Объектно-ориентированное программирование

6 семестр
Лекция 2.

Содержание лекции:

- Эволюция технологий Desktop
- Язык разметки XAML и XML
- Архитектурный паттерн MVVM
- Реализация MVVM (Интерфейсы и Команды)
- Практический пример

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Эволюция технологий Desktop. Windows Forms

Windows Forms — классическая технология для создания графических интерфейсов (GUI) в среде .NET. Является удобной «оберткой» над низкоуровневым Win32 API.

Год рождения: ~2002, вместе с .NET Framework.

•**Плюсы:**

- Очень низкий порог входа.
- Быстрое создание простых приложений для Windows.
- Огромная база знаний и готовых решений.

•**Минусы:**

- Только для Windows.
- Внешний вид привязан к версии Windows. Сложно кастомизировать.
- «Спагетти-код». Вся логика пишется внутри обработчика нажатия кнопки (button_Click). Это нарушает принципы ООП (единственная ответственность).

Windows Presentation Foundation (WPF)

WPF — современная графическая подсистема .NET для создания визуально насыщенных Windows-приложений. Внешний вид описывается на языке **XAML**, а бизнес-логика пишется на **C#**.

Год рождения: 2006 (в составе .NET Framework 3.0).

Ключевые особенности и отличия:

1. **Векторная графика:** Интерфейс не зависит от разрешения экрана (не «пикселизуется» при масштабировании).
2. **Аппаратное ускорение:** В отличие от WinForms (GDI+), WPF использует **DirectX**. Это позволяет легко работать с анимациями, 2D/3D графикой и мультимедиа.
3. **Гибкая стилизация:** Мощная система стилей и шаблонов (аналог CSS в вебе), позволяющая полностью изменить вид любого элемента.
4. **Связывание данных (Data Binding):** Продвинутый механизм синхронизации данных между кодом и интерфейсом.

Avalonia

Avalonia — это мощный open-source фреймворк для создания кроссплатформенных графических интерфейсов на базе .NET.

Наследник WPF: Построен на принципах WPF и UWP, что делает переход на него максимально легким для .NET-разработчиков.

Год появления: ~2013 (активное развитие началось с выходом .NET Core).

Кроссплатформенность – свойство программного обеспечения, позволяющее ему работать с несколькими операционными системами или аппаратными платформами без изменения исходного кода.

Ключевые особенности:

Разделение ответственности: Полная поддержка связки **XAML** (интерфейс) + **C#** (логика), что обеспечивает чистоту и гибкость кода.

Векторная графика: Интерфейс отрисовывается через собственную систему визуализации. Результат: идеальная четкость на любых экранах и ОС.

Кроссплатформенность:

- **Desktop (основное):** Единый код для **Windows, Linux и macOS**.
- **Mobile & Web:** Экспериментальная поддержка iOS, Android и браузера (WebAssembly).

Open Source: Распространяется под свободной лицензией MIT.

Язык разметки XML и XAML

XML (eExtensible Markup Language) — расширяемый язык разметки для хранения и передачи структурированных данных.

Опубликован: 1998 г.

Главная особенность: Отсутствие фиксированных тегов. Разработчик сам определяет имена элементов исходя из предметной области.

Назначение:

Универсальный формат обмена данными между разными системами.

Человекочитаемая структура.

Машинная независимость.

XML имеет строгую иерархическую структуру.

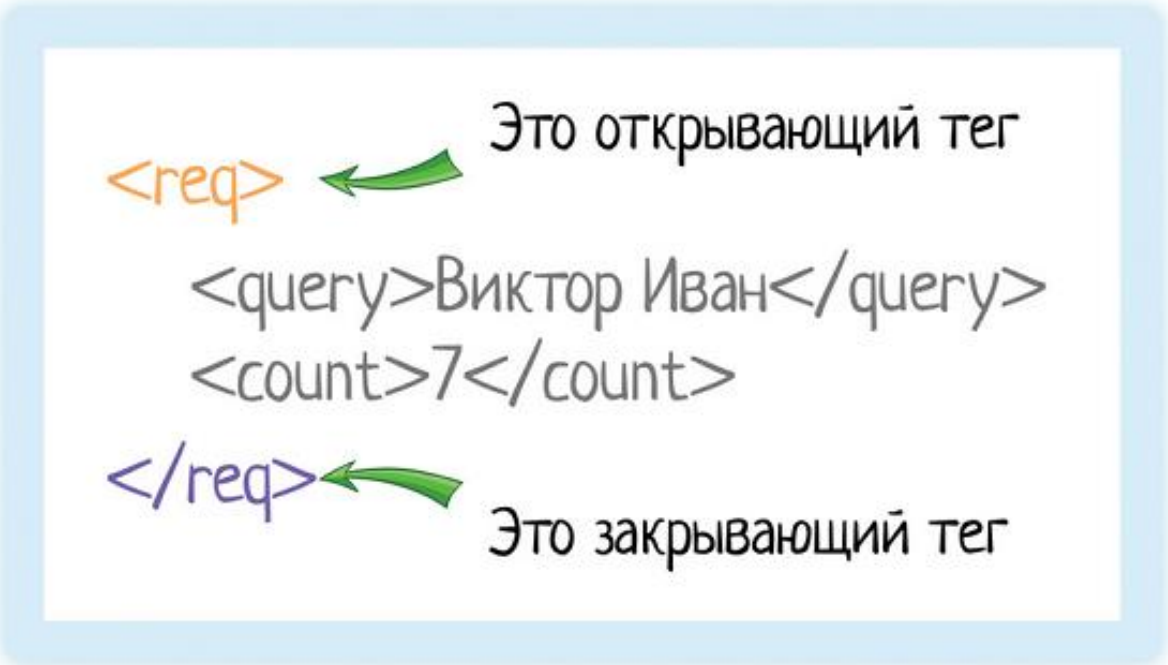
1. Пролог (необязательно): Указывает версию и кодировку.

<?xml version="1.0" encoding="UTF-8"?>

2. Корневой элемент: Обязательный «родительский» тег, в который вложены все остальные данные.

3. Элементы и Теги:

Данные заключаются в открывающий и закрывающий теги: <name>Иван</name>.



4. Атрибуты:

Дополнительные свойства элементов в формате имя="значение".

Позволяют уточнить характеристики без создания новых вложенных элементов.

Атрибуты элемента party

```
<party type="PHYSICAL" sourceSystem="AL" rawId="2">
    <field name="name">Олег </field>
    <field name="birthdate">02.01.1980</field>
    <attribute type="PHONE" rawId="AL.2.PH.1">
        <field name="type">MOBILE</field>
        <field name="number">+7 916 9754221</field>
    </attribute>
</party>
```

Пример

```xml
<?xml version="1.0" encoding="UTF-8"?>
<contactList>
    <contact>
        <firstName>Иван</firstName>
        <lastName>Иванов</lastName>
        <phone>+71234567890</phone>
        <email>ivanov@example.com</email>
    </contact>
    <contact>
        <firstName>Анна</firstName>
        <lastName>Петрова</lastName>
        <phone>+70987654321</phone>
        <email>petrova@example.com</email>
    </contact>
</contactList>
```

XAML

XAML — язык на базе XML, созданный Microsoft для проектирования пользовательских интерфейсов.

Суть: Разделение дизайна и логики. Дизайнер описывает внешний вид в XAML, программист пишет логику на C#.

Ключевые возможности:

- **Декларативность:** Вы описываете *что* должно быть на экране (кнопка, слайдер), а не *как* его рисовать.
- **Стилизация:** Мощная поддержка стилей (аналог CSS).
- **Data Binding:** Продвинутая синхронизация данных между интерфейсом и кодом.

Пример структуры:

```
<StackPanel Orientation="Horizontal">
  <Button Content="Нажать" Width="100"/>
  <TextBox Text="{Binding UserName}"/>
</StackPanel>
```

XML: Описывает **информацию**. Например, список студентов, настройки программы или каталог товаров. Ему всё равно, как это будет выглядеть на экране.

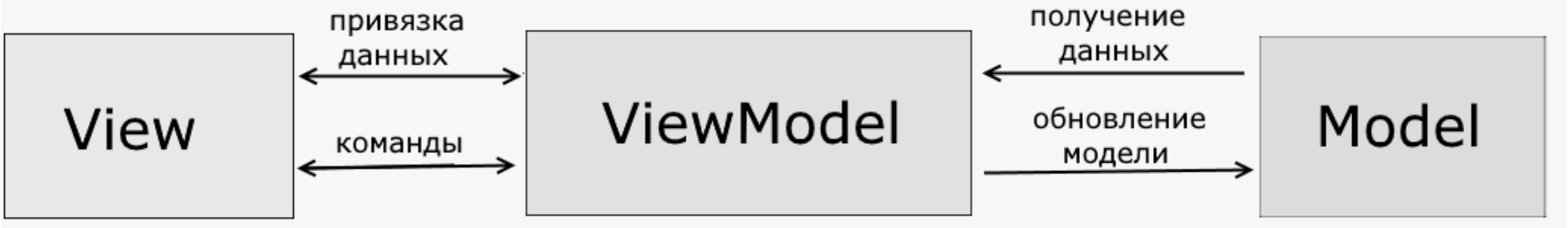
XAML: Описывает **объекты интерфейса**. Каждый тег в XAML — это реальный программный объект (кнопка, текстовое поле, окно), который появится на экране.

Архитектурный паттерн MVVM

MVVM (Model-View-ViewModel) — это архитектурный паттерн (шаблон), разделяющий визуальную часть приложения и его логику. Был представлен в 2005 году и стал стандартом для WPF, Avalonia, Android и iOS.

Автор: Джон Госсман (John Gossman, Microsoft)

Данный паттерн является **архитектурным**, то есть он задает общую архитектуру приложения.



Основные компоненты:

•**Model (Модель):** Данные и бизнес-логика (например, класс User или запросы к БД). «Чистый» код, который ничего не знает об интерфейсе.

•**View (Представление):** Визуальный интерфейс (XAML). «Глупая» часть системы: она только отображает данные и не содержит логики расчетов.

•**ViewModel (Модель представления):** Связующее звено и «мозг» интерфейса.

- Забирает данные из **Модели**.
- Преобразует их в удобный для отображения вид.
- Обрабатывает команды от **View** (например, нажатие кнопки).

Главная особенность MVVM — отсутствие прямой связи между кодом и интерфейсом. Взаимодействие происходит через «магию» **привязок**.

Data Binding (Привязка данных): Механизм автоматической синхронизации между **View** и **ViewModel**.

Изменили значение переменной в коде — текст на экране обновился сам.

Пользователь ввел текст в поле — значение в коде изменилось автоматически.

Команды (Commands): Вместо классических обработчиков событий (как в WinForms), View отправляет во ViewModel команду на выполнение действия.

В чем профит?

Независимость: Программист может писать логику, а дизайнер — менять интерфейс в XAML одновременно, не мешая друг другу.

Тестируемость: Логику во ViewModel можно протестировать даже без запуска графического окна.

Чистота: Код интерфейса не перемешан с бизнес-задачами.

Реализация MVVM (Интерфейсы и Команды)

Проблема: В C# обычные свойства «молчат». Если вы измените User.Name в коде, текст в текстовом поле на экране останется прежним, потому что интерфейс не следит за памятью ежесекундно.

Решение: Интерфейс INotifyPropertyChanged (INPC)

Это стандартный механизм «оповещения» в .NET, который делает объекты разговорчивыми.

Суть: Класс ViewModel реализует этот интерфейс, предоставляя событие PropertyChanged.

Механика: Когда значение свойства меняется, объект должен буквально **«крикнуть»** интерфейсу: *«Эй, моё свойство 'Имя' изменилось, перерисуй меня!»*.

Результат: View «слышит» этот сигнал и мгновенно обновляет именно тот элемент, который привязан к этому свойству.

```
public class MainWindowViewModel : INotifyPropertyChanged
{
    private string? _userName;

    // Свойство для TextBox и TextBlock
    public string? UserName
    {
        get => _userName;
        set
        {
            _userName = value;
            OnPropertyChanged(); // Уведомляем интерфейс
        }
    }

    // Стандартная реализация уведомлений
    public event PropertyChangedEventHandler? PropertyChanged;
    protected void OnPropertyChanged(string? name = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}
```

Проблема: В старых подходах (WinForms) код писался в «обработчиках событий» (например, Button_Click). Это намертво привязывало логику к конкретной кнопке и конкретному окну.

Решение: Паттерн «Команда» (ICommand) – В MVVM мы не обрабатываем «клик», мы вызываем «Команду».

Суть: Вместо события Click, свойство Command у кнопки привязывается к объекту типа ICommand во ViewModel.

Почему это круто:

Тестируемость: Вы можете запустить логику нажатия кнопки из обычного Unit-теста, даже не создавая само окно приложения.

Универсальность: Одну и ту же команду можно привязать к кнопке, пункту меню и горячей клавише одновременно.

Управление состоянием: Команда умеет говорить кнопке, можно ли её нажимать в данный момент (метод CanExecute). Если команда «неактивна», кнопка на экране заблокируется автоматически.

```
public class MainWindowViewModel : INotifyPropertyChanged
{
    // Команда для Button
    public ICommand SaveCommand { get; }

    public MainWindowViewModel()
    {
        // Инициализируем команду действием
        SaveCommand = new RelayCommand(() =>
        {
            // Здесь ваша логика при нажатии на кнопку
            Debug.WriteLine($"Сохранено имя: {UserName}");
        });
    }
}
```

```
using System;
using System.Windows.Input;

namespace AvaloniaApplication1;

// Простая реализация ICommand, чтобы кнопка работала
public class RelayCommand : ICommand
{
    private readonly Action _execute;
    public RelayCommand(Action execute) => _execute = execute;
    public bool CanExecute(object? parameter) => true;
    public void Execute(object? parameter) => _execute();
    public event EventHandler? CanExecuteChanged;
}
```

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        mc:Ignorable="d"
        d:DesignWidth="800"
        d:DesignHeight="450"
        x:Class="AvaloniaApplication1.MainWindow"

        xmlns:vm="using:AvaloniaApplication1"
        x:DataType="vm:MainWindowViewModel"

        Title="AvaloniaApplication1">

</Window>
```

```
        <!-- View: наше окно -->
        <StackPanel>
            <!-- Текстовое поле слушает свойство UserName -->
            <TextBox Text="{Binding UserName,
UpdateSourceTrigger=PropertyChanged}" />

            <!-- Кнопка нажимает Команду, а не событие Click --
>
            <Button Content="Сохранить изменения"
                    Command="{Binding SaveCommand}"
                    Width="200" />

            <!-- Текст дублируется автоматически благодаря
уведомлениям -->
            <TextBlock Text="{Binding UserName}" />
        </StackPanel>
```

```
        <!-- View: наше окно -->
        <StackPanel>
            <!-- Текстовое поле слушает свойство UserName -->
            <TextBox Text="{Binding UserName,
UpdateSourceTrigger=PropertyChanged}" />

            <!-- Кнопка нажимает Команду, а не событие Click --
>
            <Button Content="Сохранить изменения"
                    Command="{Binding SaveCommand}"
                    Width="200" />

            <!-- Текст дублируется автоматически благодаря
уведомлениям -->
            <TextBlock Text="{Binding UserName}" />
        </StackPanel>
```

```
using Avalonia.Controls;

namespace AvaloniaApplication1
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // ВОТ ЭТА СТРОЧКА — МАГИЯ СВЯЗИ:
            DataContext = new MainWindowViewModel();
        }
    }
}
```

Пример. Reactive UI

Reactive UI (RxUI) — это фреймворк, построенный на основе реактивного программирования (Reactive Extensions, или Rx), который позволяет создавать приложения, реагирующие на изменения данных, используя наблюдаемые последовательности (Observables).

Зачем он нужен?

Упрощение MVVM: Избавляет от рутинного кода (вызовов INotifyPropertyChanged, команд).

Мощные привязки: Позволяет привязывать не только свойства, но и целые потоки событий.

Управление состоянием: Упрощает обработку асинхронных операций, валидации и сложных взаимодействий между компонентами.

```
public class MainWindowViewModel : ReactiveObject
{
    private string _message = "Добро пожаловать в Avalonia + ReactiveUI!";
    private int _count;

    public MainWindowViewModel()
    {
        // Создаем вычисляемое свойство, которое реагирует на изменения
Count
        this.WhenAnyValue(x => x.Count)
            .Select(count => $"Количество нажатий: {count}")
            .ToProperty(this, x => x.DisplayCount, out _displayCount);

        // Инициализация команд
        IncrementCommand = ReactiveCommand.Create(IncrementCount);

        // Команда сброса активна только когда счетчик > 0
        ResetCommand = ReactiveCommand.Create(ResetCount,
            this.WhenAnyValue(x => x.Count).Select(count => count > 0));
    }
```

```
// Reactive свойства
public string Message
{
    get => _message;
    set => this.RaiseAndSetIfChanged(ref _message, value);
}

public int Count
{
    get => _count;
    set => this.RaiseAndSetIfChanged(ref _count, value);
}

// Вычисляемое свойство через Observable
private readonly ObservableAsPropertyHelper<string> _displayCount;
public string DisplayCount =>
                _displayCount?.Value ?? "Количество нажатий: 0";
```

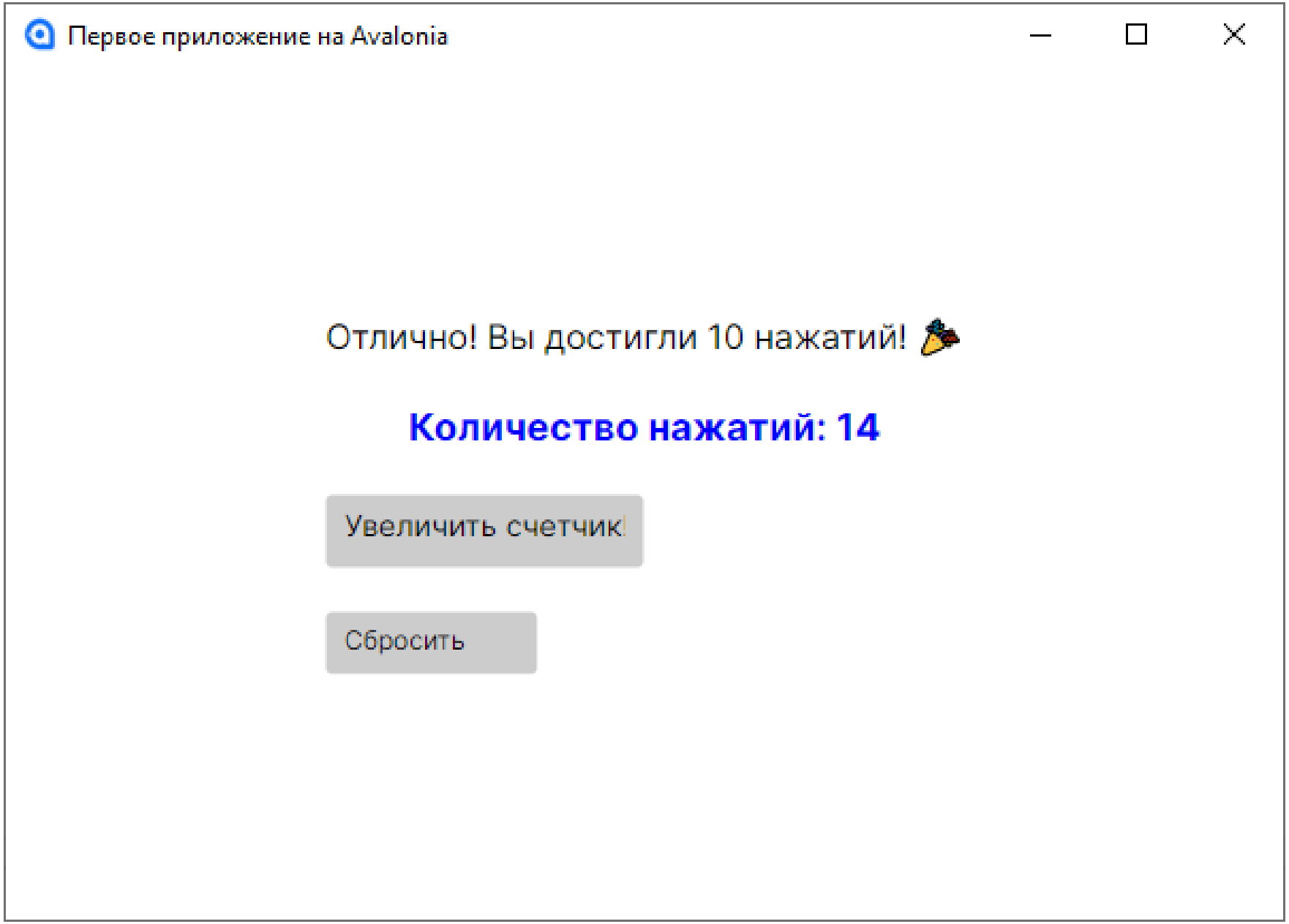
```
    // Команды
    public ReactiveCommand<Unit, Unit> IncrementCommand { get; }
    public ReactiveCommand<Unit, Unit> ResetCommand { get; }

    private void IncrementCount()
    {
        Count++;

        if (Count % 10 == 0)
            Message = $"Отлично! Вы достигли {Count} нажатий!";
    }

    private void ResetCount()
    {
        Count = 0;
        Message = "Счетчик сброшен! Давайте начнем снова.";
    }
}
```

View с ReactiveUI

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:vm="using:AvaloniaApplication1.ViewModels"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" Width="600" Height="400"
        x:Class="AvaloniaApplication1.Views.MainWindow"
        x:DataType="vm:MainWindowViewModel"
        Icon="/Assets/avalonia-logo.ico"
        Title="Первое приложение на Avalonia">

    <Design.DataContext>
        <vm:MainWindowViewModel/>
     </Design.DataContext>

        <!- Далее -->

</Window>
```

```
<StackPanel HorizontalAlignment="Center"
            VerticalAlignment="Center" Spacing="20">

    <!-- Приветственный текст -->
    <TextBlock Text="{Binding Message}"
               FontSize="16"
               TextAlignment="Center"
               HorizontalAlignment="Center"/>

    <!-- Счетчик с привязкой к вычисляемому свойству -->
    <TextBlock Text="{Binding DisplayCount}"
               FontSize="18"
               FontWeight="Bold"
               Foreground="Blue"
               HorizontalAlignment="Center"/>
```

```
        <!-- Кнопки с привязкой команд ReactiveUI -->
        <Button Content="Увеличить счетчик!"
                Command="{Binding IncrementCommand}"
                Width="150" Height="35"
                FontSize="14"/>

        <Button Content="Сбросить"
                Command="{Binding ResetCommand}"
                Width="100" Height="30"
                FontSize="12"/>

    </StackPanel>
```

Замечание: Чтобы отображалась кириллица необходимо, чтобы расширение файла было UTF-8.

Пример работы программы